Cross-Site Scripting (XSS)

1. Objašnjenje klase napada (XSS)

Cross-Site Scripting (XSS) je web bezbednosna ranjivost koja omogućava napadaču da kompromituje interakciju korisnika sa ranjivom aplikacijom. U praksi to znači da aplikacija na nebezbedan način obradi ili prikaže podatke koje napadač kontroliše, pa browser žrtve izvrši zlonamerni JavaScript kao da je deo legitimne stranice.

Na taj način napadač može da zaobiđe zaštitu koju pruža same-origin policy i da deluje u kontekstu sesije žrtve.

Tri glavna oblika XSS napada su:

- reflected XSS – zlonamerni unos dolazi iz trenutnog HTTP zahteva i odmah se vraća u odgovoru aplikacije;

- stored XSS – zlonamerni unos se čuva u aplikaciji, a zatim se kasnije prikazuje drugim korisnicima;

- DOM-based XSS – ranjivost postoji u klijentskom JavaScript kodu, koji nebezbedno obrađuje podatke iz nepouzdanog izvora i upisuje ih u DOM.

Kako XSS funkcioniše:

Napadač ubaci payload koji sadrži JavaScript ili HTML/JavaScript konstrukciju pogodnu za konkretan kontekst. Ako aplikacija taj unos vrati ili obradi bez odgovarajuće zaštite, kod se izvršava u browseru korisnika i dobija ista prava kao legitimni skriptovi te aplikacije.

2. Uticaj napada (Impact)

Posledice XSS napada zavise od same aplikacije, vrste podataka koje obrađuje i privilegija korisnika čija je sesija kompromitovana.

Napadač najčešće može:

- da se predstavi kao žrtva korisnik i obavlja radnje u njegovo ime;

- da izvrši sve akcije koje žrtva inače sme da izvrši u aplikaciji;

- da pročita podatke kojima žrtva ima pristup;

- da prikupi pristupne podatke, kao što su korisnička imena i lozinke;

- da vizuelno izmeni sadržaj stranice ili ubaci dodatnu, trojansku funkcionalnost.

U aplikacijama koje sadrže javne i neosetljive podatke uticaj može biti manji. Međutim, u sistemima koji sadrže finansijske podatke, e-mail komunikaciju, zdravstvene informacije ili administratorske funkcije, posledice mogu biti veoma ozbiljne. Ako je kompromitovan korisnik sa povišenim

privilegijama, napadač može praktično preuzeti kontrolu nad većim delom aplikacije i podacima drugih korisnika.

3. Koje ranjivosti omogućavaju napad

XSS nastaje kada aplikacija prihvati podatke iz nepouzdanog izvora i prikaže ih ili obradi na nebezbedan način. Najčešći uzroci su:

- vraćanje korisničkog unosa u HTTP odgovoru bez odgovarajućeg kodiranja izlaza;

- čuvanje korisničkog unosa u bazi ili drugom skladištu, pa njegovo kasnije prikazivanje bez zaštite;

- upotreba opasnih DOM operacija, kao što je upis nepouzdanog sadržaja pomoću innerHTML ili sličnih sinkova;

- nedovoljno stroga validacija ulaza, posebno kada aplikacija prihvata HTML ili druge kompleksne formate;

- nepravilno rukovanje različitim izlaznim kontekstima, na primer u HTML-u, atributima, URL-ovima ili JavaScript kodu.

Primer DOM-based XSS situacije:

```
var search = document.getElementById('search').value;
var results = document.getElementById('results');
results.innerHTML = 'You searched for: ' + search;
```

Ako napadač kontroliše vrednost promenljive search, a aplikacija je bezbednosno ne obradi pre upisa u DOM, može izazvati izvršavanje proizvoljnog JavaScript koda.

4. Kontramere (zaštita)

Efikasna zaštita od XSS-a obično zahteva kombinaciju više mera, jer jedna mera sama po sebi najčešće nije dovoljna.

1. Filtriranje i validacija ulaza

Prilikom prijema korisničkog unosa treba što strože dozvoliti samo ono što je očekivano i validno. To je posebno važno za polja sa unapred poznatim formatom.

2. Kodiranje izlaza

Kada se korisnički kontrolisani podaci prikazuju u odgovoru aplikacije, potrebno ih je pravilno kodirati kako browser ne bi taj sadržaj protumačio kao aktivan kod. Vrsta kodiranja zavisi od konteksta: HTML, atribut, URL, JavaScript ili CSS.

3. Bezbedno rukovanje DOM-om

Treba izbegavati rizične operacije koje ubacuju nepouzdan sadržaj direktno u DOM. Prednost treba dati bezbednijim API-jima i template mehanizmima koji automatski escape-uju izlaz.

4. Odgovarajući HTTP zaglavlja

Za odgovore koji ne treba da sadrže HTML ili JavaScript važno je pravilno postaviti Content-Type i X-Content-Type-Options, kako bi browser sadržaj tumačio na očekivan način.

5. Content Security Policy (CSP)

CSP može služiti kao poslednja linija odbrane i smanjiti posledice XSS ranjivosti koje ipak ostanu u aplikaciji. Ipak, CSP ne treba posmatrati kao zamenu za ispravnu validaciju i kodiranje.

Zadaci

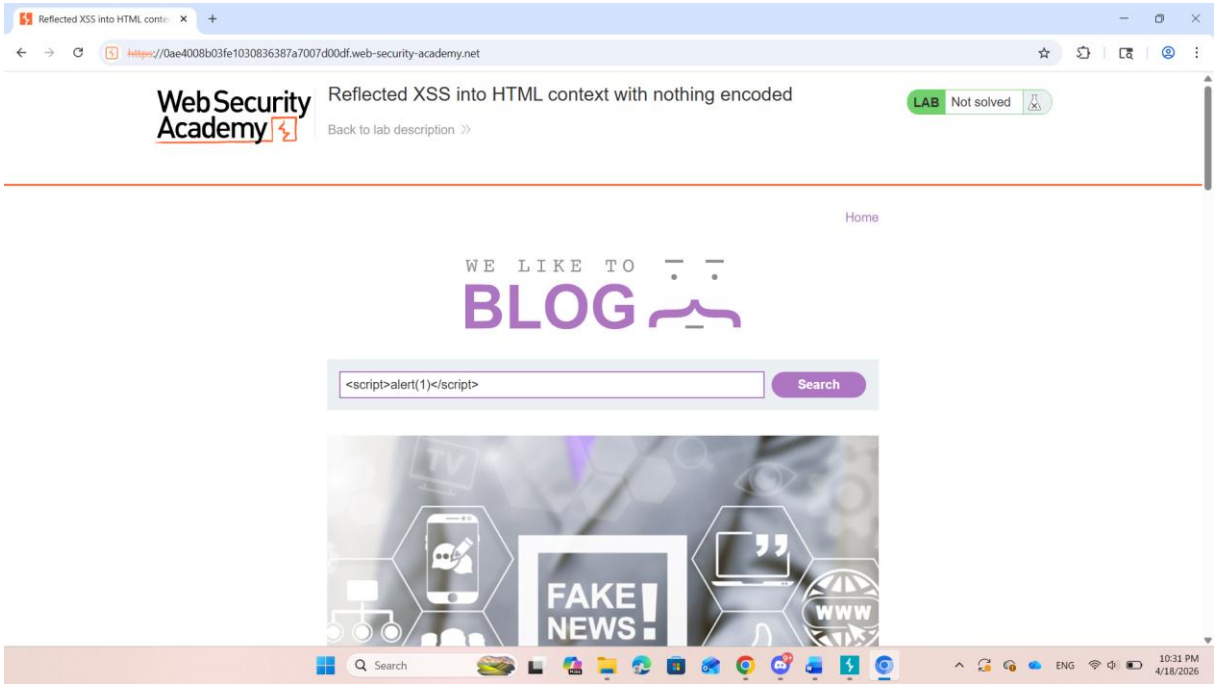
Writeup zadatka – Reflected XSS into HTML context with nothing encoded

Opis zadatka: Cilj laboratorijske vežbe bio je da se iskoristi reflected XSS ranjivost u polju za pretragu bloga tako da se JavaScript payload vrati u HTML odgovoru bez ikakvog kodiranja i izvrši u browseru korisnika.

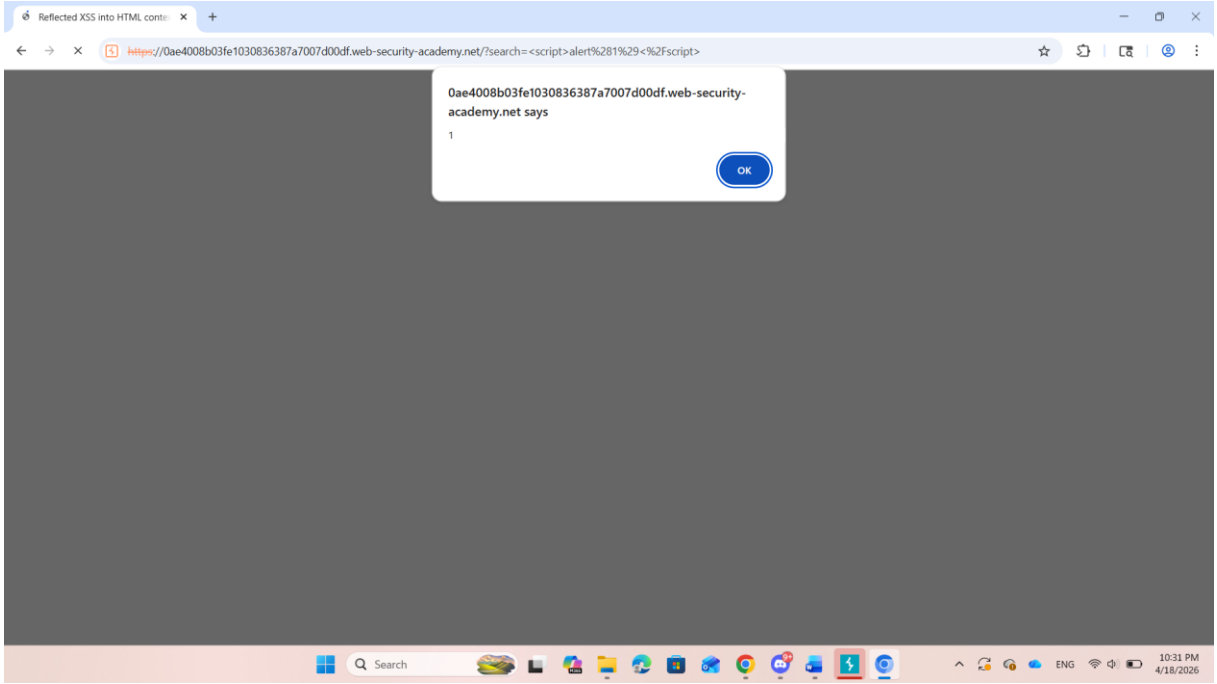
- Korak 1. Otvoren je lab „Reflected XSS into HTML context with nothing encoded“ i prikazana je početna stranica bloga sa poljem za pretragu.
- Korak 2. U polje za pretragu unet je payload "<script>alert(1)</script>" i zatim je pokrenuta pretraga klikom na dugme Search.
- Korak 3. Nakon slanja zahteva aplikacija je vrednost parametra search vratila direktno u HTML odgovor stranice rezultata, bez ikakvog kodiranja specijalnih karaktera.
- Korak 4. Browser je reflektovani unos protumačio kao JavaScript kod, zbog čega se izvršio payload alert(1) i prikazan je alert dijalog sa vrednošću 1. Time je potvrđeno da je XSS ranjivost uspešno iskorišćena.

Objašnjenje payload-a: Tag <script> omogućava ubacivanje JavaScript koda u HTML dokument, dok izraz alert(1) služi kao jednostavna potvrda izvršavanja skripta. Pošto aplikacija nije escape-ovala znakove < i > niti je na drugi način neutralisala unos, browser je prosleđeni payload izvršio kao aktivan kod umesto da ga prikaže kao običan tekst.

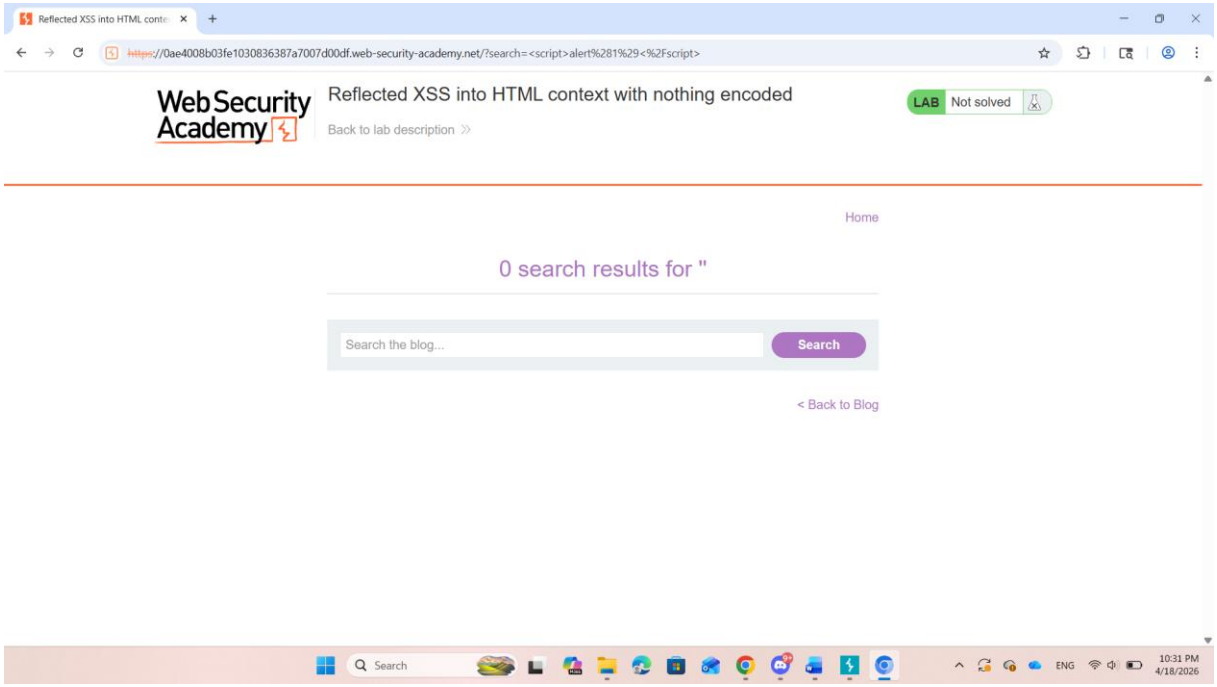
Slika 1. Početna stranica laboratorijske vežbe pre ubacivanja payload-a.



Slika 2. Unos payload-a <script>alert(1)</script> u polje za pretragu bloga.



Slika 3. Izvršavanje payload-a i prikaz alert dijaloga u browseru.



Slika 4. Stranica rezultata pretrage nakon reflektovanja unosa i izvršavanja XSS payload-a.

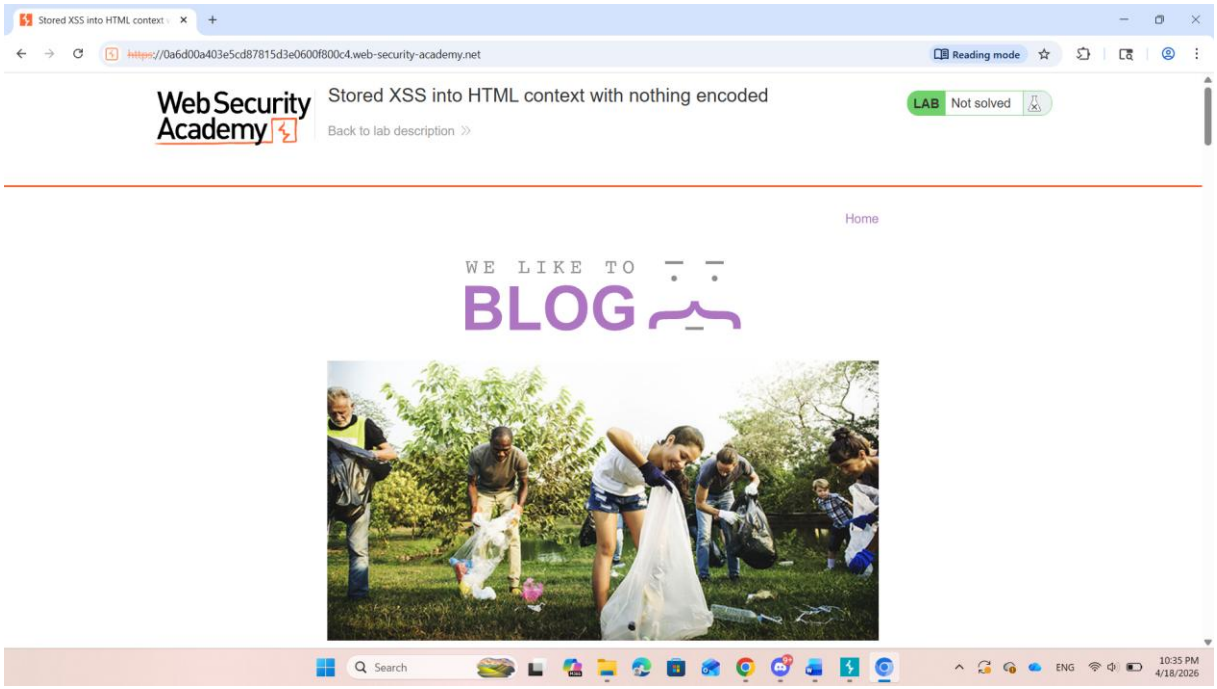
Zaključak: Zadatak je rešen ubacivanjem JavaScript payload-a u parametar pretrage i potvrdom njegovog izvršavanja u browseru. Ovo pokazuje da aplikacija korisnički unos vraća direktno u HTML odgovor bez odgovarajućeg output encoding-a, što omogućava izvršavanje proizvoljnog JavaScript koda u kontekstu aplikacije.

Writeup zadatka – Stored XSS into HTML context with nothing encoded

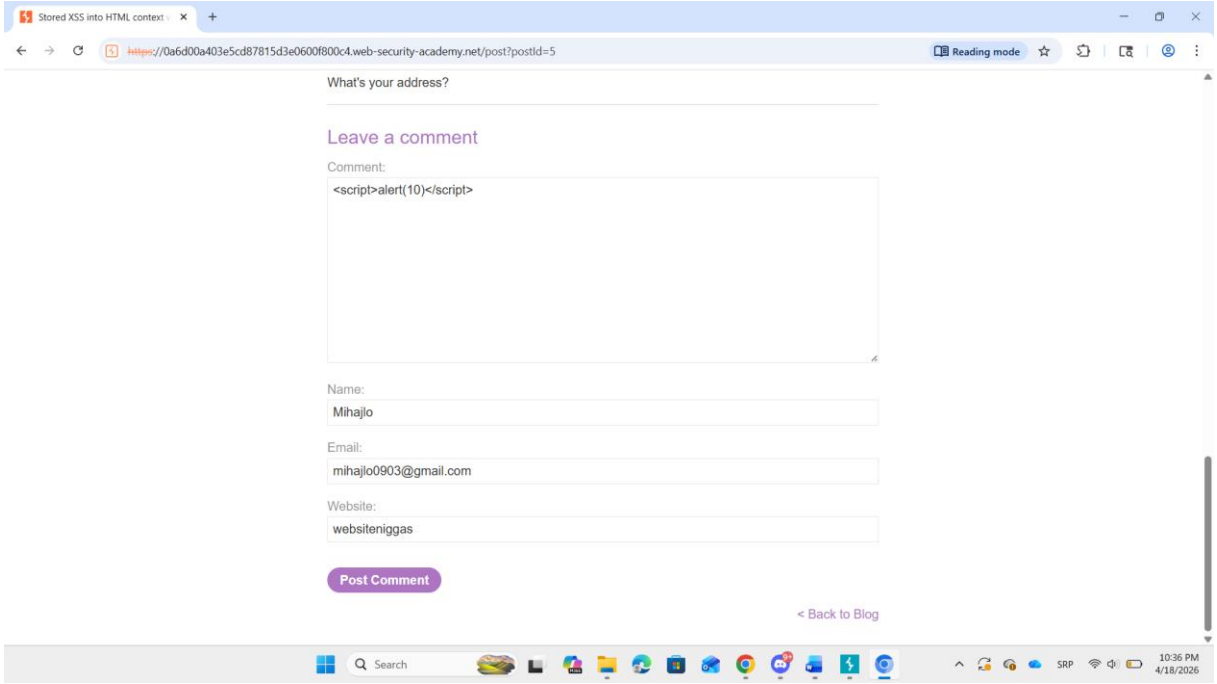
Opis zadatka: Cilj laboratorijske vežbe bio je da se iskoristi stored XSS ranjivost u formi za ostavljanje komentara tako da se JavaScript payload sačuva u aplikaciji i zatim izvrši prilikom prikaza komentara u browseru.

- Korak 1. Otvoren je lab „Stored XSS into HTML context with nothing encoded“ i izabran je blog post sa formom za unos komentara.
- Korak 2. U polje Comment unet je payload "<script>alert(10)</script>", dok su preostala obavezna polja popunjena proizvoljnim vrednostima kako bi forma mogla da bude poslata.
- Korak 3. Nakon klika na dugme Post Comment aplikacija je prihvatila komentar i sačuvala uneti sadržaj bez ikakvog kodiranja ili neutralizacije script taga. Time je potvrđeno da je zlonamerni unos uspešno uskladišten.
- Korak 4. Kada je stranica posta prikazala sačuvani komentar, browser je sadržaj komentara protumačio kao JavaScript kod i izvršio payload alert(10), čime je potvrđena stored XSS ranjivost i zadatak je uspešno rešen.

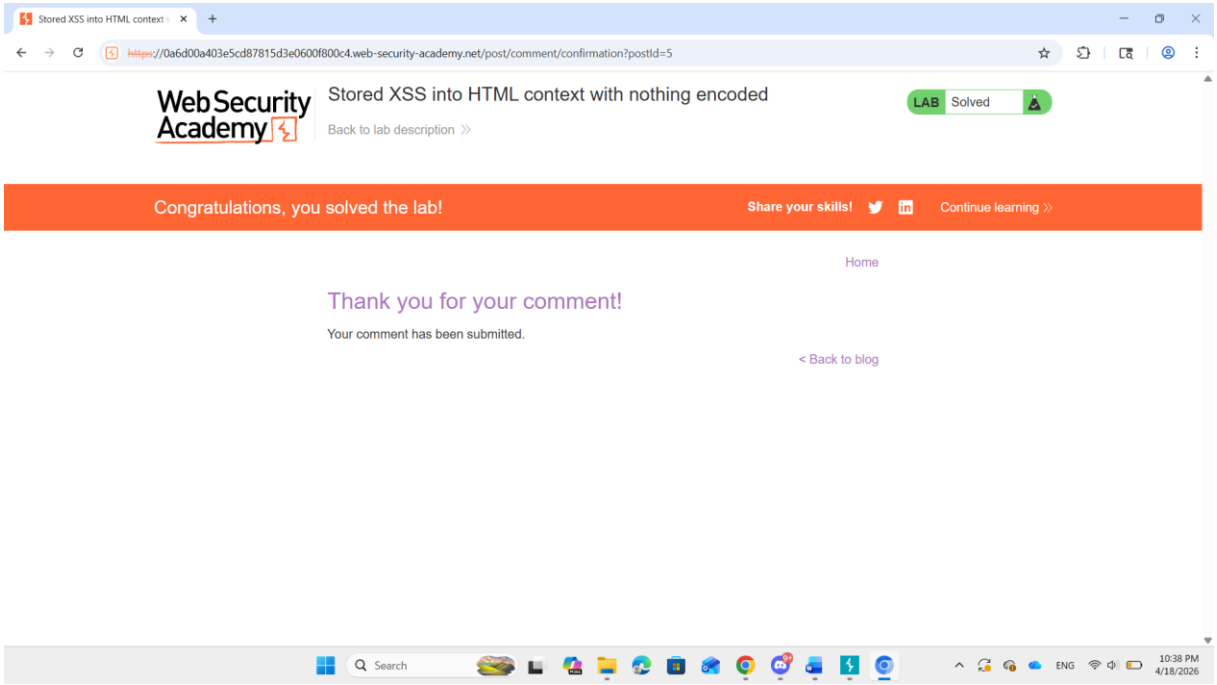
Objašnjenje payload-a: Tag <script> ubacuje JavaScript kod u HTML dokument, dok izraz alert(10) služi kao jednostavna potvrda da se skript izvršava. Pošto aplikacija komentar čuva i kasnije prikazuje bez odgovarajućeg output encoding-a, browser prilikom renderovanja stranice izvršava sačuvani payload umesto da ga prikaže kao običan tekst.



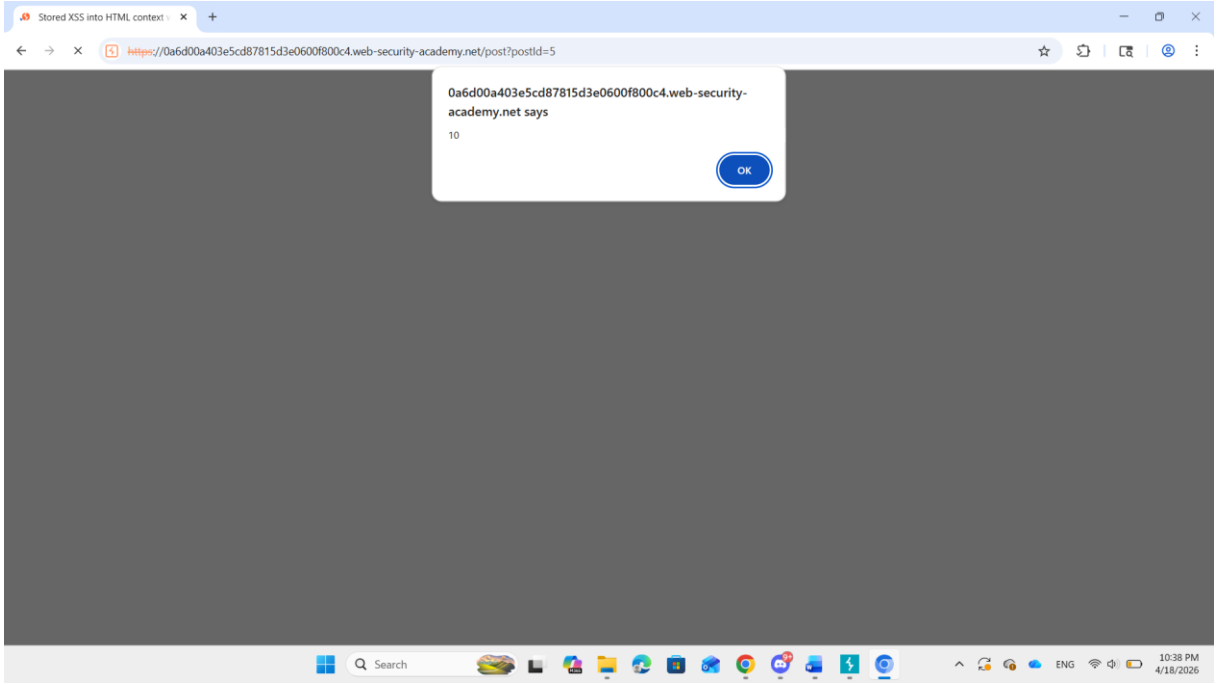
Slika 5. Početna stranica laboratorijske vežbe sa prikazom blog posta.



Slika 6. Unos payload-a <script>alert(10)</script> u formu za ostavljanje komentara.



Slika 7. Potvrda da je komentar prihvaćen i da je laboratorijska vežba uspešno rešena.



Slika 8. Izvršavanje stored XSS payload-a i prikaz alert dijaloga pri učitavanju stranice posta.

Zaključak: Zadatak je rešen ubacivanjem JavaScript payload-a u polje za komentar i potvrdom njegovog izvršavanja nakon što je komentar sačuvan i prikazan na stranici. Ovo pokazuje da aplikacija korisnički unos trajno čuva i kasnije vraća u HTML bez odgovarajuće zaštite, što omogućava izvršavanje proizvoljnog JavaScript koda svaki put kada se kompromitovani sadržaj prikaže.

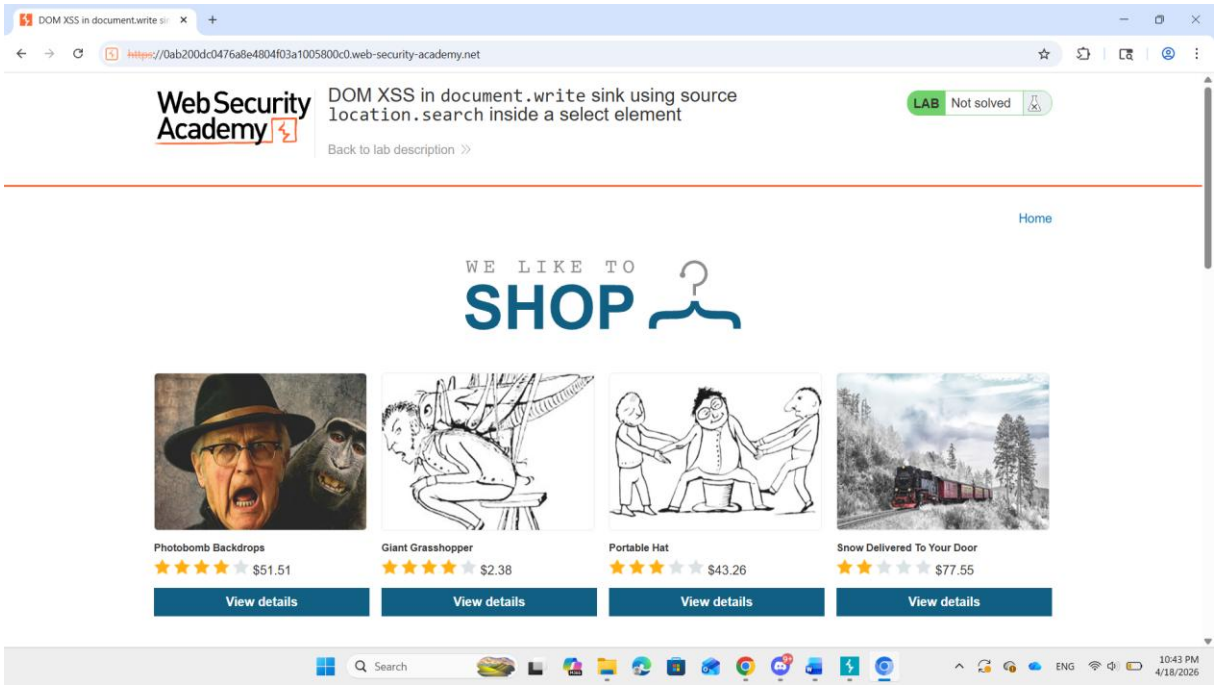
Writeup zadatka – DOM XSS in document.write sink using source location.search inside a select element

Opis zadatka: Cilj laboratorijske vežbe bio je da se iskoristi DOM-based XSS ranjivost u kojoj klijentski JavaScript uzima vrednost iz location.search i pomoću funkcije document.write upisuje je unutar select elementa. Zadatak je uspešno rešen ubacivanjem posebno oblikovanog payload-a u parametar storeId, čime je izvršen JavaScript kod u browseru.

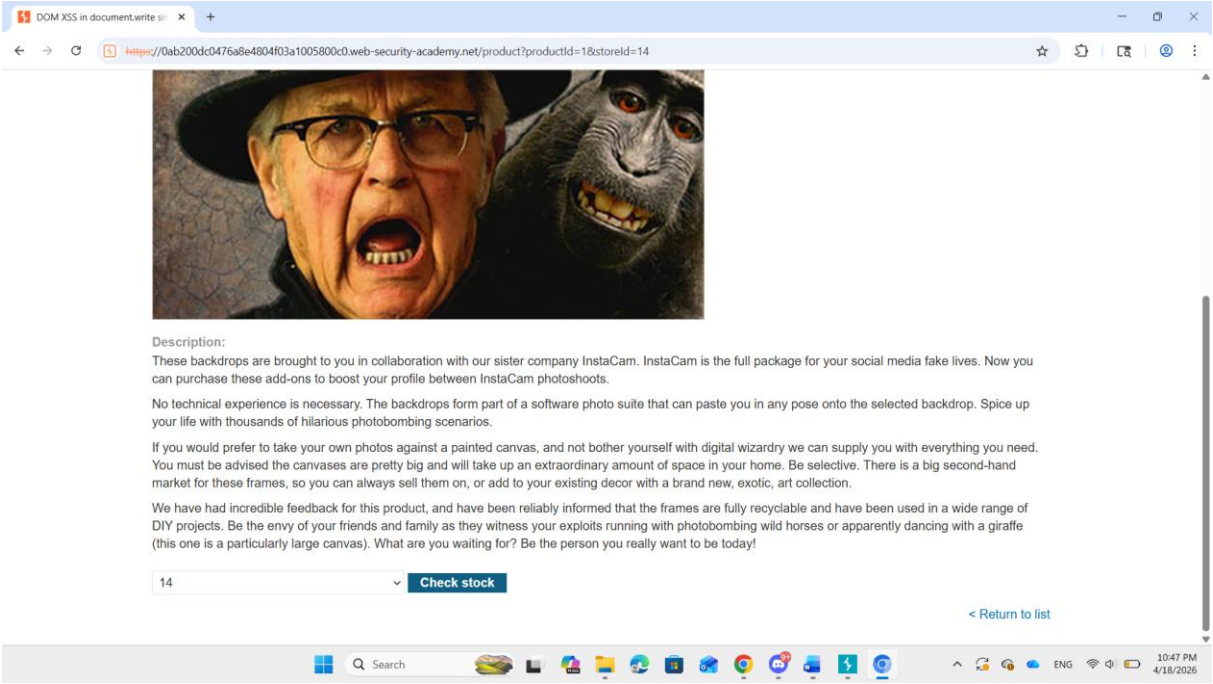
- Korak 1. Otvoren je lab „DOM XSS in document.write sink using source location.search inside a select element“, a zatim je sa početne stranice izabran proizvod i otvorena njegova detaljna stranica.
- Korak 2. Na stranici proizvoda pregledan je HTML/DOM u Developer Tools alatu i uočeno je da aplikacija vrednost parametra storeId iz URL-a upisuje pomoću document.write unutar select elementa za izbor prodavnice.
- Korak 3. Parametar storeId u URL-u je zatim zamenjen payload-om `"></select><img src=1 onerror=alert(1)>`, čime je postojeći select kontekst zatvoren i ubačen novi HTML element sa JavaScript događajem onerror.

- Korak 4. Nakon učitavanja izmenjenog URL-a browser je ubaceni img element obradio kao deo stranice. Pošto resurs nije mogao da se učita, aktivirao se događaj onerror i izvršio se payload alert(1), čime je potvrđena DOM XSS ranjivost.

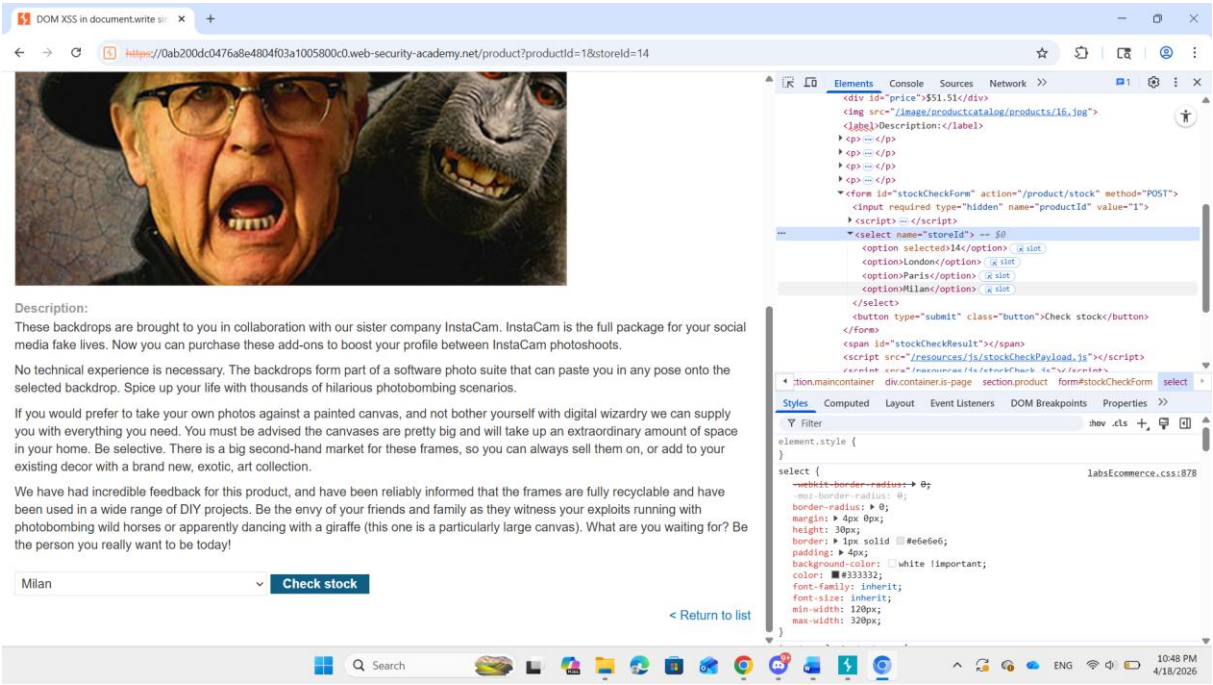
Objašnjenje payload-a: Sekvenca `"></select>` služi za izlazak iz postojećeg select elementa u koji aplikacija upisuje korisnički kontrolisan sadržaj. Nakon toga se ubacuje element `<img src=1 onerror=alert(1)>`. Pošto slika sa izvorom 1 ne može uspešno da se učita, browser aktivira događaj onerror i izvršava JavaScript kod `alert(1)`. Upravo zato je ovaj payload pogodan za DOM XSS situacije u kojima je unos smešten unutar HTML strukture, a ne direktno unutar script taga.



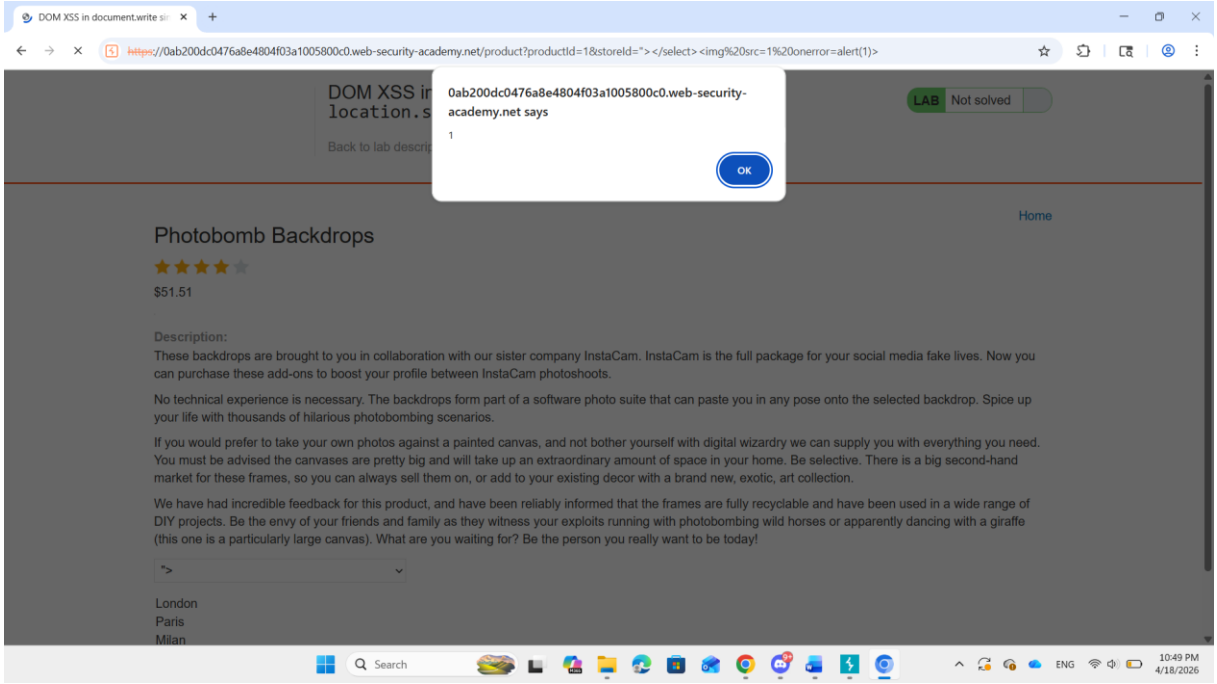
Slika 9. Početna stranica laboratorijske vežbe pre otvaranja detalja proizvoda.



Slika 10. Stranica proizvoda sa regularnom vrednošću parametra storeId pre ubacivanja payload-a.



Slika 11. Pregled DOM strukture u Developer Tools alatu i prikaz select elementa u koji se vrednost storeId upisuje pomoću document.write.



Slika 12. Izvršavanje DOM XSS payload-a i prikaz alert dijaloga u browseru nakon izmene URL parametra storeId.

Zaključak: Zadatak je rešen manipulacijom URL parametra koji klijentski JavaScript nebezbedno obrađuje i ubacuje u DOM pomoću document.write. Ovo pokazuje da DOM XSS ne nastaje nužno na serverskoj strani, već i u browseru, kada se nepouzdan sadržaj iz location.search koristi bez odgovarajućeg escape-ovanja i bezbedne DOM obrade.

Writeup zadatka – DOM XSS in AngularJS expression with angle brackets and double quotes HTML-encoded

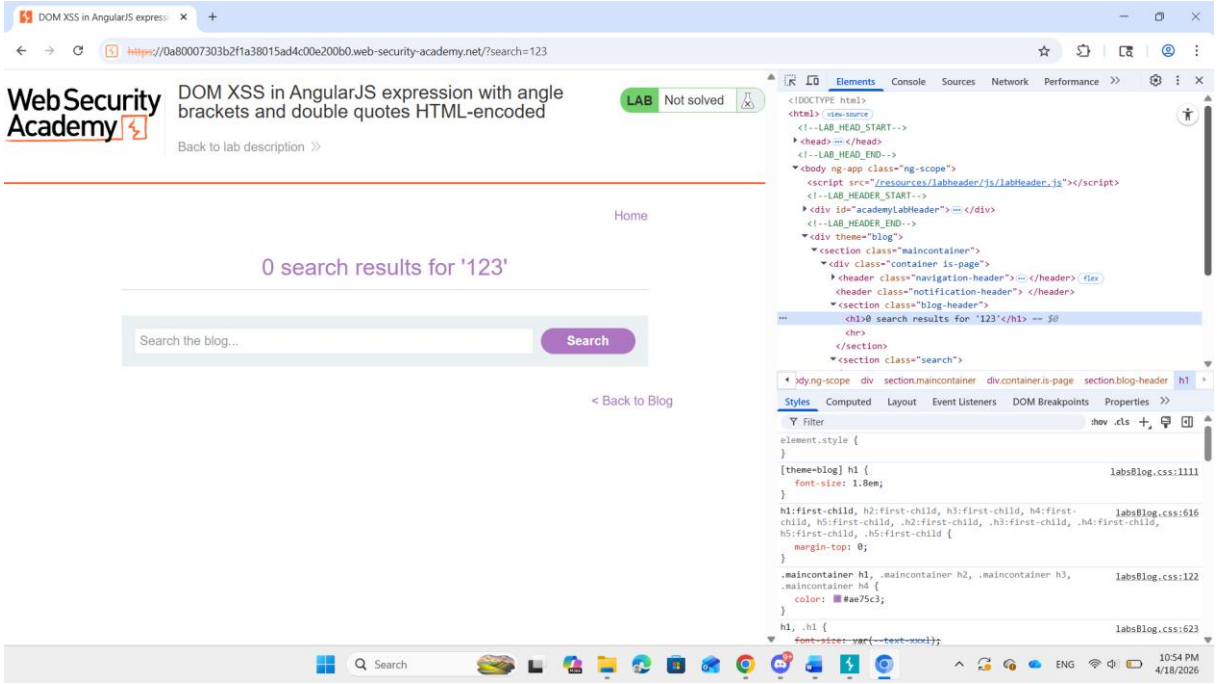
Opis zadatka: Cilj laboratorijske vežbe bio je da se iskoristi DOM XSS ranjivost u AngularJS okruženju, iako su znakovi <, > i navodnici HTML-encoded. Zadatak je zahtevao da se pronađe način da se reflektovani unos ipak izvrši kao AngularJS izraz i da se time potvrdi izvršavanje JavaScript koda u browseru.

- Korak 1. Otvoren je lab „DOM XSS in AngularJS expression with angle brackets and double quotes HTML-encoded“ i najpre je izvršena obična pretraga sa vrednošću `123` kako bi se proverilo gde se korisnički unos prikazuje na stranici.
- Korak 2. Nakon toga je uz pomoć Developer Tools alata pregledan DOM i uočeno je da se reflektovana vrednost pretrage nalazi unutar stranice koja koristi AngularJS, što potvrđuje atribut `ng-app` na body elementu. Takođe je prikazano da se unesena vrednost pojavljuje u naslovu rezultata pretrage.
- Korak 3. Pošto su uglaste zagrade i navodnici HTML-encoded, klasičan HTML payload nije bio pogodan za iskorišćavanje ranjivosti. Zbog toga je u parametar `search` unet AngularJS payload `{{$on.constructor('alert(1)')()}}`, koji je zatim URL-enkodovan i prosleđen aplikaciji.

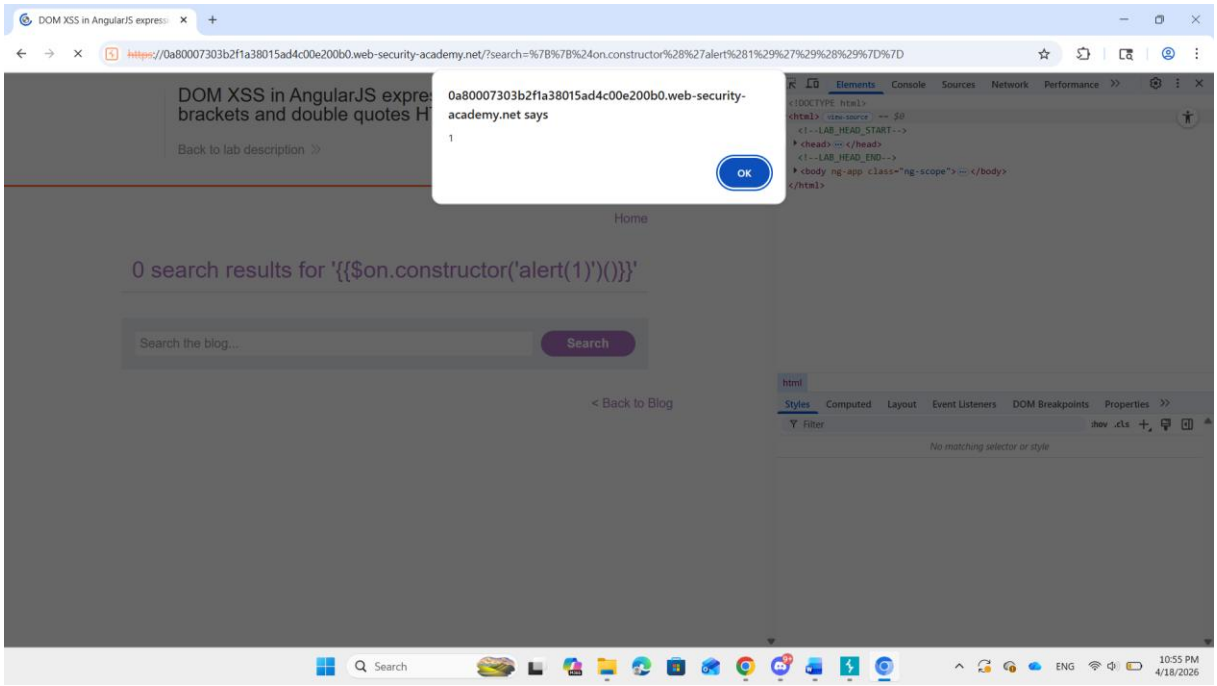
- Korak 4. Nakon učitavanja izmenjenog URL-a AngularJS je obradio uneseni izraz, konstruisao i izvršio JavaScript kod, što je potvrđeno prikazom `alert(1)` dijaloga u browseru. Time je potvrđeno da je laboratorijska vežba uspešno rešena.

Objašnjenje payload-a: Izraz `{{$on.constructor('alert(1)')()}}` koristi činjenicu da je `$on` funkcija dostupna u AngularJS kontekstu. Pristupom njenoj osobini `constructor` dobija se Function konstruktor, kojim se formira funkcija sa sadržajem `alert(1)`, a zatim se ona odmah izvršava pomoću završnih zagrada `()`. Na taj način payload ne zavisi od HTML tagova, već od evaluacije AngularJS izraza.

Slika 13. Početna stranica laboratorijske vežbe pre unošenja payload-a.



Slika 14. Prikaz reflektovane vrednosti pretrage u DOM-u i potvrda da stranica koristi AngularJS okruženje.



Slika 15. Izvršavanje AngularJS payload-a i prikaz alert dijaloga u browseru nakon izmene parametra search.

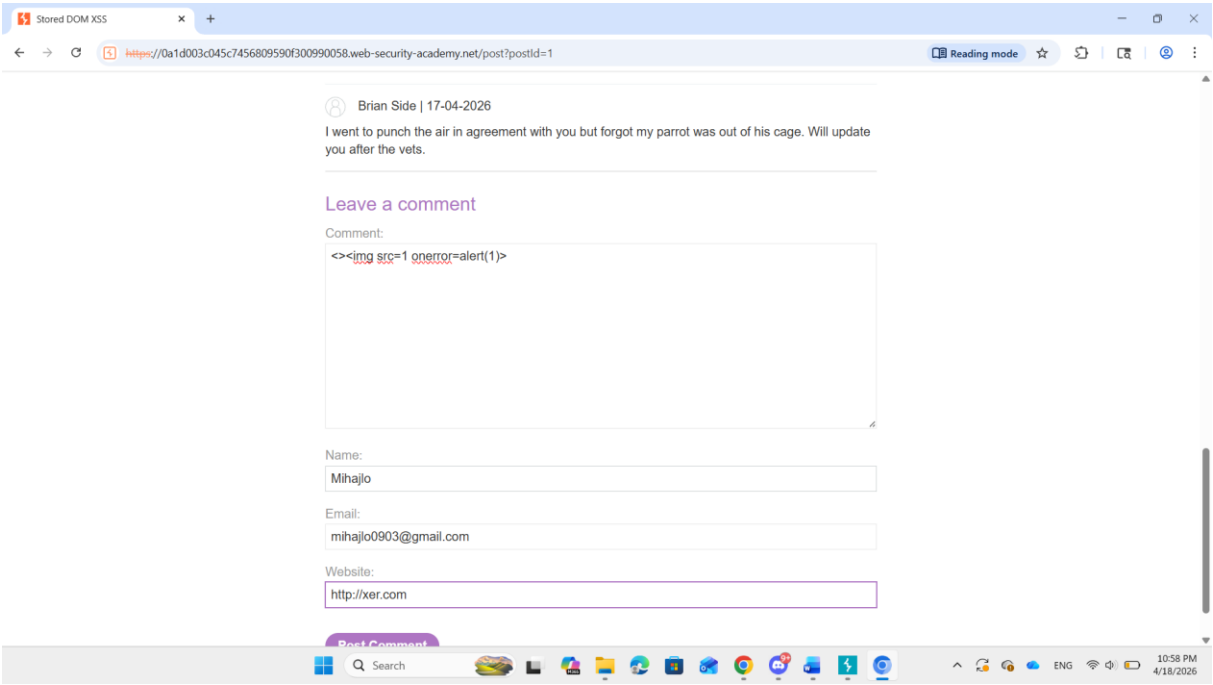
Zaključak: Zadatak je rešen iskorišćavanjem AngularJS izraza umesto klasičnog HTML ili JavaScript ubacivanja. Ovo pokazuje da DOM XSS može nastati i kada su određeni karakteri HTML-encoded, ako aplikacija reflektovani unos prosleđuje framework-u koji ga dalje interpretira kao izvršiv izraz.

Writeup zadatka – Stored DOM XSS

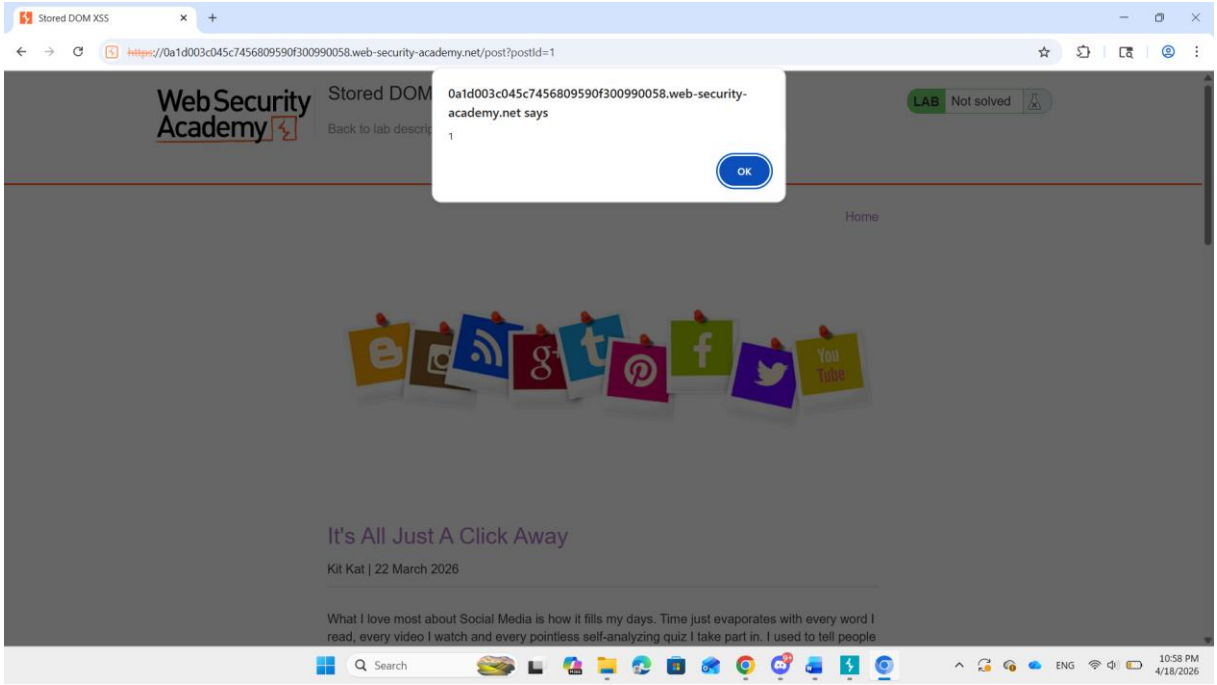
Opis zadatka: Cilj laboratorijske vežbe bio je da se iskoristi stored DOM XSS ranjivost u funkcionalnosti za komentare. Zadatak je zahtevao da se zlonamerni HTML/JavaScript payload sačuva kroz formu za komentar, a zatim da se njegovo izvršavanje potvrdi kada aplikacija kasnije prikaže sačuvani sadržaj u DOM-u stranice blog posta.

- Korak 1. Otvoren je lab „Stored DOM XSS“ i izabran je blog post sa formom za ostavljanje komentara kako bi se uneli test podaci i pripremio napad.
- Korak 2. U polje Comment unet je payload `<><img src=1 onerror=alert(1)>`, dok su polja Name, Email i Website popunjena proizvoljnim vrednostima kako bi aplikacija prihvatila komentar.
- Korak 3. Nakon slanja forme aplikacija je sačuvala komentar. Pri kasnijem prikazu sačuvanog sadržaja u okviru stranice posta, browser je obradio ubačeni HTML element kao deo DOM-a.
- Korak 4. Pošto je vrednost atributa `src` namerno neispravna, učitavanje slike nije uspelo, pa se aktivirao `onerror` handler i izvršio JavaScript kod `alert(1)`, čime je potvrđena stored DOM XSS ranjivost.

Objašnjenje payload-a: Sekvenca `<><img src=1 onerror=alert(1)>` ubacuje HTML `img` element u sadržaj komentara. Pošto resurs naveden u atributu `src` ne postoji, browser aktivira događaj `onerror`, a u njemu se izvršava JavaScript izraz `alert(1)`. Time se potvrđuje da aplikacija kasnije prikazuje sačuvani korisnički unos na način koji omogućava izvršavanje koda u DOM-u.



Slika 16. Unos payload-a `<><img src=1 onerror=alert(1)>` u formu za ostavljanje komentara.



Slika 17. Izvršavanje stored DOM XSS payload-a i prikaz alert dijaloga u browseru.

Slika 18. Početna stranica laboratorijske vežbe „Stored DOM XSS“ pre izvođenja napada.

Zaključak: Zadatak je rešen skladištenjem zlonamernog payload-a kroz formu za komentare i potvrdom njegovog izvršavanja prilikom kasnijeg prikaza sadržaja u browseru. Ovo pokazuje da stored XSS ne mora da nastane samo direktnim serverskim ubacivanjem u HTML odgovor, već i onda kada aplikacija sačuvane podatke naknadno nebezbedno obrađuje i upisuje u DOM.